ШУТИС — МЭДЭЭЛЭЛ ХОЛБООНЫ ТЕХНОЛОГИЙН СУРГУУЛЬ

F.CSM311 — Программ хангамжийн бүтээлт

Лабораторийн ажил №12

Бат бөх дизайн ба алдаа зохицуулалт (Designing for Robustness)

Хугацаа: 90 минут

Хичээлийн код	F.CSM311
Лекцтэй холбоо	Лекц 12 — Designing for Robustness
Програмчлалын хэл	Java 17+
Бүтээх хэрэгсэл	Maven 3.6+, JUnit 5
Оролт	lab12-starter.zip
Гаралт	lab12-<овог>-<нэр>.zip (ажиллаж буй тестүүдтэй)

1. Лабораторийн зорилго

Энэхүү лабораторийн ажлын зорилго нь Лекц 12-т танилцсан бат бөх (robust) программ зохиох зарчмуудыг бодит Java код дээр практикаар хэрэгжүүлэх явдал юм. Оюутан дараах зүйлсийг гардан хийж сурна:

- Fail-fast зарчим ба Design by Contract (precondition, postcondition, invariant) ашиглах
- null-ийг Optional болон Null Object Pattern-ээр халах
- Exception-ыг зөв залгихгүй (swallow), try-with-resources ашиглах, төрлийг орчуулах (translation)
- Exponential backoff + jitter-тэй retry логик бичих
- 3 төлөвт (CLOSED/OPEN/HALF_OPEN) Circuit Breaker паттерн хэрэгжүүлэх
- Fault injection тестээр алдааны замыг (failure path) баталгаажуулах

2. Урьдчилсан мэдлэг

- Java үндсэн синтакс, generic, lambda, Supplier<T>
- JUnit 5 ашиглан тест бичиж, assert хийх
- Лекц 11-ийн API дизайны зарчмууд (өмнө үзсэн)
- Maven-ээр төсөл ажиллуулах (mvn test)

3. Хугацааны задаргаа (90 минут)

Хугацаа	Үе шат	Агуулга
0–5 мин	Бэлтгэл	lab12-starter.zip-ыг татаж, IDE-д нээж, mvn test ажиллуулах
5–20 мин (15')	Даалгавар 1	BankAccount — fail-fast, preconditions, InsufficientFundsException
20–35 мин (15')	Даалгавар 2	UserRepository-г Optional болгох + NullLogger pattern
35–50 мин (15')	Даалгавар 3	FileConfigLoader — try-with-resources, exception translation
50–70 мин (20')	Даалгавар 4	RetryExecutor — exponential backoff + jitter
70–85 мин (15')	Даалгавар 5	SimpleCircuitBreaker — 3 төлөвт хэрэгжилт
85–90 мин (5')	Хүлээлгэх	Бүх тест ногоон байгааг баталж, zip үүсгэн илгээх

Нэмэлт (bonus): Хугацаа үлдвэл Даалгавар 4-т RetryExecutor-ыг SimpleCircuitBreaker-тэй хослуулж, үйлчилгээ давхардан уналт өгсөн тохиолдолд fallback утга буцаах жижиг dahин дасгалыг бич.

4. Төслийг бэлтгэх

1. lab12-starter.zip файлыг ClassHub/Moodle-оос татаж, дурын фолдерт задлана.
2. IDE (IntelliJ IDEA, Eclipse, VS Code)-доо 'Open Maven Project' сонголтоор нээнэ.
3. Терминалаас дараах командыг ажиллуулан бүх зависимтэй тестүүдийг ачаалж үзнэ:

```
cd lab12-starter
mvn test
```

Анх удаа ажиллахад бүх тест унана — энэ нь хэвийн үзэгдэл. Таны үүрэг нь тус бүр TODO-г бөглөж тестүүдийг ногоон болгох явдал.

Даалгавар 1 (15 минут) — Fail-fast ба Preconditions

Контекст

Банкны данстай ажиллах BankAccount класс өгөгдсөн. Одоогийн хэрэгжилт нь ямар ч оролтын шалгалт хийхгүй тул доорх асуудлууд гарч байна:

- null owner-той данс үүсгэж болно
- Сөрөг эхний үлдэгдэл зөвшөөрөгдөж байна
- withdraw нь сөрөг дүнг хүлээн авч, invariant (balance ≥ 0)-ыг зөрчиж байна
- transfer нь NullPointerException-аар уналт өгч болзошгүй

Зорилт

src/main/java/mn/csm311/lab12/task1/BankAccount.java файл дахь бүх TODO тэмдэглэгээг fail-fast зарчмын дагуу бөглө.

Шаардлагууд

- Precondition зөрчигдвөл IllegalArgumentException (Java-гийн стандарт) шидэх
- Хангалтгүй үлдэгдэл бол тусгайлан үүсгэсэн InsufficientFundsException шидэх
- Invariant-ыг assert-ээр баталгаажуулах (assert balance >= 0)
- transfer статик аргыг withdraw + deposit-ээс бүрдүүлж, withdraw амжилтгүй бол deposit хийгдэхгүй байхаар бичих

Амжилтын шалгуур

```
mvn test -Dtest=BankAccountTest


Expected: Tests run: 9, Failures: 0, Errors: 0
```

Зөвлөмж

- Java 11+ дээр String.isBlank() туслана: if (s == null || s.isBlank()) …
- transfer-д from == to-г === (==) лавлагааны харьцуулалтаар шалгана
- withdraw амжилттай болсны дараа л deposit хийнэ — дэс дараалал чухал

Даалгавар 2 (15 минут) — Optional ба Null Object Pattern

Контекст

Tony Hoare null-ийг 'billion-dollar mistake' гэж нэрлэсэн. Java-д NullPointerException бол хамгийн түгээмэл алдаануудын нэг. Энэ даалгавраар null-ийг тип системд 'бол бараг'-тай болгож, дотоод код цэвэр болгоно.

2a. UserRepository-г Optional-д хөрвүүлэх

task2/UserRepository.java дахь findByEmail нь User буцаахын оронд Optional<User> буцаадаг болго. Optional.ofNullable(...) туслана.

2b. NullLogger хэрэгжүүлэх

task2/NullLogger.java нь Logger интерфэйсийг 'юу ч хийхгүй' байдлаар хэрэгжүүлнэ. log() нь юу ч хэвлэхгүй, logCount() нь үргэлж 0 буцаана. Ингэснээр 'if (logger != null)' гэх шалгалтгүй болно.

2c. GreetingService-ийг Optional API-ээр бич

task2/GreetingService.java дахь greet() аргыг findByEmail-ын буцаасан Optional<User>-ийг .map(...).orElse(...) гинжээр ашиглаж бич.

Хүлээгдэж буй гаралт

```
greet("bat@example.com")  // => "Сайн байна уу, Bat!"
greet("missing@example.com")  // => "Сайн байна уу, Зочин!"
```

Амжилтын шалгуур

```
mvn test -Dtest=UserRepositoryTest  # Tests run: 6, Failures: 0
```

Даалгавар 3 (15 минут) — Exception handling

Контекст

task3/FileConfigLoader.java файл key=value хэлбэрийн тохиргооны файлыг унших болович 4 ноцтой асуудалтай: (A) Exception-ыг залгиж (swallow) байна, (B) finally блок доор close()-д дахин алдаа гарч болзошгүй, (C) IOException нь дотоод хэрэгжилтийн дэлгэрэнгүй болж гадаад API луу гадагшилж байна, (D) 'файл олдсонгүй' онцгой тохиолдлыг ерөнхий IOException-оос ялгаатай мессежээр мэдээлдэггүй.

Зорилт

load(Path path) аргыг дараах байдлаар эргэн бич:

4. path == null бол IllegalArgumentException шидэх (fail-fast)
5. BufferedReader-ыг try-with-resources-д нээх
6. NoSuchFileException барьж "config file not found: <path>" мессежтэй ConfigLoadException шидэх
7. Үлдсэн IOException-ыг "failed to read config: <path>" мессежтэйгээр орчуулах
8. cause-ыг алдалгүй дамжуулах (super(message, cause))

Жишээ шийдлийн хэв маяг

```
try (BufferedReader r = Files.newBufferedReader(path)) {
    …
} catch (NoSuchFileException e) {
    throw new ConfigLoadException("config file not found: " + path, e);
} catch (IOException e) {
    throw new ConfigLoadException("failed to read config: " + path, e);
}
```

Амжилтын шалгуур

```
mvn test -Dtest=FileConfigLoaderTest  # Tests run: 3, Failures: 0
```

Даалгавар 4 (20 минут) — Retry with Exponential Backoff + Jitter

Контекст

Сүлжээний 'түр зуурын' (transient) алдаа нь ихэвчлэн дахин оролдож засагдана. Гэхдээ хуулан оролдлого нь thundering herd үүсгэхгүй байхаар exponential backoff + jitter-тэй байх ёстой. Мөн зарим алдаа (жишээ: 400 Bad Request) нь retry хийх ёсгүй — эдгээрийг NonRetryableException болгон тэмдэглэсэн.

Зорилт

task4/RetryExecutor.java-гийн execute(Supplier<T> op) аргыг дараах алгоритмыг дагаж хэрэгжүүл:

```
for attempt = 1 .. maxAttempts:
    try:
        return op.get()
    catch NonRetryableException e:
        throw e               # шууд дамжуулна
    catch RuntimeException e:
        lastError = e
        if attempt == maxAttempts: break
        delay = baseDelayMs * 2^(attempt-1) + random(0..baseDelayMs)
        sleeper.sleep(delay)
throw lastError
```

Гол санаанууд

- Exponential: 100ms → 200ms → 400ms → 800ms … олон клиент нэгэн зэрэг сэргэхээс сэргийлнэ
- Jitter: санамсаргүй 0..baseDelayMs мөс нэмж хэмнэлтийг таслана
- Сүүлийн оролдлогын дараа унтдаггүй — энэ нь тестүүдэд чухал
- Sleeper интерфэйсийг FakeSleeper-ээр солиж, тест хурдан ажиллана

Амжилтын шалгуур

```
mvn test -Dtest=RetryExecutorTest  # Tests run: 5, Failures: 0
```

Нийтлэг алдаа

- InterruptedException-ыг залгилгүй, Thread.currentThread().interrupt() дуудах
- Оролдлого болгонд биш, оролдлогын ХООРОНД л унтах — сүүлчийн хэрэглэхгүй
- lastError нь try/catch-ын хамрах хүрээнээс гадна зарлагдсан байх ёстой

Даалгавар 5 (15 минут) — Circuit Breaker

Контекст

Цахилгааны автомат таслуур шиг — хэт их алдаа гарахад Circuit Breaker 'тасалдаг' ба удсан үйлчилгээнд дахин дахин оролдож, thread дүүсгах, cascading failure үүсгэхээс сэргийлдэг.

3 төлөв

Төлөв	Тайлбар
CLOSED	Ихэнх хүсэлт нэвтэрнэ. Дараалсан N алдаа → OPEN
OPEN	Бүх хүсэлт шууд fast-fail. resetTimeout өнгөрмөгц → HALF_OPEN
HALF_OPEN	Нэг хүсэлт туршина. Амжилттай → CLOSED, алдаатай → OPEN

Зорилт

task5/SimpleCircuitBreaker.java-гийн execute, onSuccess, onFailure аргуудыг хэрэгжүүл. state() арга нь OPEN→HALF_OPEN шилжилтийг автоматаар хийдэг — үүнийг ашиглана.

Гол төлөвийн шилжилт

- execute() дотор: хэрэв state() == OPEN → CircuitBreakerOpenException
- onSuccess: state = CLOSED, failureCount = 0
- onFailure @ HALF_OPEN: state = OPEN, openedAt = clock.now(), failureCount = 0
- onFailure @ CLOSED: failureCount++; ≥ failureThreshold бол → OPEN

Тестийн хиймэл цаг

Clock интерфэйсийг FakeClock-оор солихоор бичсэн тул бодит үнтлагагүйгээр цагийн шилжилтийг тест хийх боломжтой. Энэ нь deterministic, хурдан тестийн чухал арга.

Амжилтын шалгуур

```
mvn test -Dtest=SimpleCircuitBreakerTest  # Tests run: 7, Failures: 0
```

5. Бүх тестийг ажиллуулах

Бүх даалгаврыг дуусгасны дараа өмнөхөө шалгах:

```
mvn test
```

Хүлээгдэж буй үр дүн:

```
[INFO] Tests run: 30, Failures: 0, Errors: 0, Skipped: 0
[INFO] BUILD SUCCESS
```

6. Үнэлгээний шалгуур (нийт 100 оноо)

Хэсэг	Оноо	Үзүүлэлт
Даалгавар 1	20	BankAccount-ийн 9 тест ногоон, fail-fast дүрмийг дагасан
Даалгавар 2	15	Optional, NullLogger, GreetingService-ийн 6 тест ногоон
Даалгавар 3	15	try-with-resources + exception translation, 3 тест ногоон
Даалгавар 4	25	Retry логик, backoff+jitter, NonRetryable handle, 5 тест ногоон
Даалгавар 5	20	CircuitBreaker 3 төлөв зөв ажиллана, 7 тест ногоон
Кодын чанар	5	Уншигдах нэршил, тохирох комментууд, IDE warning байхгүй

Нэмэлт оноо (+10): RetryExecutor + SimpleCircuitBreaker-ыг хослуулсан 'ResilientClient' ангийг жижиг fault-injection тесттэй хамт нэмж ирүүлэх.

7. Хүлээлгэн өгөх заавар

9. target/, .idea/, .vscode/, *.iml фолдеруудыг устгана.
10. Төслийг zip болгоно: lab12-<овог>-<нэр>.zip
11. Moodle/ClassHub-д хичээлийн дэд хэсэгт хүлээлгэж өгнө.
12. Төгсгөлт хугацаа: дараагийн лекц эхлэхээс 24 цагийн өмнө.

Ёс зүйн санамж

Өөр оюутны код хуулбарлах, GitHub/Stack Overflow-ээс шууд copy-paste хийх нь академик шударгаас зөрчил юм. Concept-ыг ойлгосон бол өөрийн үгээр шийдэж чадна.

8. Нэмэлт уншлага

- Michael Nygard — Release It! (Production хэв маягийн сурах бичиг)
- Google SRE Book — sre.google/books (бодит үйлчилгээний туршлага)
- Resilience4j documentation — CircuitBreaker-ийн production-grade хэрэгжилт
- Netflix Chaos Monkey — principlesofchaos.org
- AWS Well-Architected Framework — Reliability pillar